

IT Migration & Dependencies Brief

Nevada County Experience — technical reference for county IT staff

This document supports the technical evaluation of moving the Nevada County Experience platform onto county-owned infrastructure. It enumerates every external service the platform touches, the data flow in both directions, four hosting scenarios with cost and policy implications, a step-by-step migration plan, ongoing administrative responsibilities, and a policy-block impact matrix.

Key facts up front:

- Platform is a static HTML/JS site plus a small Python (Flask) admin server.
- No database — all data lives in JSON files inside the project directory.
- No visitor accounts, no PII collected; itinerary save is opt-in localStorage.
- Outbound network calls only — no inbound listening except HTTPS/443.
- AI categorization is optional; site is fully functional without it.
- Total monthly cost on a \$6/mo VPS: ~\$7-8 including AI use at chamber scale.

1. External Services Inventory

Every external dependency the platform touches, with vendor, purpose, data flow direction, monthly cost, what fails if the service is blocked or down, and replacement options. **Outbound** = our server initiates; **Inbound** = visitor reaches us. There are no inbound vendor connections.

Service	Purpose	Direction	Cost	Required?
Anthropic Claude API	AI categorize button — refines tags, venue, area, quality flags on scraped events	Outbound HTTPS to api.anthropic.com	\$0.30–\$1/mo	Optional
Plausible Analytics	Privacy-respecting aggregate metrics (page views, itinerary build count). No cookies, no PII	Outbound HTTPS to plausible.io	\$0 (self-host) or \$9/mo cloud	Optional, opt-in
GitHub Pages	Static-site hosting (current public deploy). Replaceable by any web server	Outbound HTTPS to github.com on deploy only	\$0	Replaceable
Google Fonts CDN	Playfair Display + Josefin Sans typefaces. Hotlinked from the index.html head section	Outbound HTTPS from visitor browser to fonts.googleapis.com	\$0	Replaceable (self-host)

Service	Purpose	Direction	Cost	Required?
Leaflet (Cloudflare CDN)	Map library for the "View on Map" feature. Hotlinked from cdnjs.cloudflare.com	Outbound HTTPS from visitor browser	\$0	Replaceable (self-host)
Unsplash CDN	Stock photography for theme cards. ~30 hotlinked images	Outbound HTTPS from visitor browser to images.unsplash.com	\$0	Replaceable (self-host)
KVMR website (scrape target)	Community-radio events calendar — major source of weekly events	Outbound HTTPS from server (kvmr.org)	\$0	Optional source
Trumba JSON feed (NCAC)	Nevada County Arts Council calendar — 540 events, 16-month window. JSON, no auth	Outbound HTTPS to trumba.com	\$0	Optional source
Eventbrite, NC Chamber, GV Chamber, The Union	Additional event scrape sources. Public HTML pages, no auth	Outbound HTTPS from server	\$0	Optional sources
Let's Encrypt	Free TLS certificates via certbot. Required for HTTPS	Outbound HTTPS to acme-v02.api.letsencrypt.org during cert renewal	\$0	Required for HTTPS
Gmail SMTP (optional alerting)	Currently used by trade-related projects; not used by this platform	N/A here	N/A	Not used

What we do not use: no third-party tracking / advertising / identity / payment / CDN-for-data services. No social-login providers. No vendor SDK or proprietary library — the entire site is plain HTML/JS that opens in any modern browser.

2. Architecture & Data Flow

2a. Components

Component	Tech	Disk path	Network
Public static site	HTML + JS (single file)	index.html	Served on :443 to visitors
Admin server	Python 3.12 + Flask	server.py	Listens on :5000 behind nginx
Event scraper	Python 3 + Selenium (Chromium headless)	scraper/event_scraper.py	Outbound HTTPS to source sites
AI categorizer	Python 3 + Anthropic SDK	scraper/ai_categorize.py	Outbound HTTPS to api.anthropic.com
Events queue file	JSON	scraper_output/events.json	Local only
Suggestions file	JSON	scraper_output/suggestions.json	Local only
Scraper sources file	JSON	scraper/sources.json	Local only
Itinerary state	localStorage in visitor's browser	n/a (client-side)	Never leaves the visitor's device

2b. Data flow

Visitor path: browser → CDN (Google Fonts, Leaflet, Unsplash, optional Plausible) and HTTPS to our server :443 → nginx reverse-proxies to Flask on :5000 → returns index.html and JSON event data. No cookies set; no server-side session.

Admin path: chamber staff browser → HTTPS to /admin (gated by either localhost-only access or ?admin=1 query parameter on a whitelisted hostname) → Flask serves the admin endpoints (events queue, suggestions, scraper trigger, AI categorize trigger).

Scheduled job path: cron / systemd timer / Windows Task Scheduler → invokes Python scraper → scraper fetches HTML/JSON from each enabled source → writes new entries to events.json → optionally invokes AI categorizer which posts to Anthropic API and writes enriched fields back to events.json.

What flows OUT of the county: only the scrape requests (public-page fetches), AI categorize prompts (event titles + descriptions, no PII), and Let's Encrypt cert renewal calls. No visitor data is ever transmitted to a third party.

3. Hosting Scenarios

Four realistic options for where the platform runs. Pick by whatever best matches county policy on cloud vs. on-prem and OS preference. Migration effort is similar across A, B, and C (Linux). Scenario D (Windows Server) is the heaviest lift because the scraper expects a POSIX environment.

3a. Scenario A — Linux VPS (DigitalOcean, Vultr, Linode)

Profile: \$6–12/month managed VPS. Ubuntu 24.04 LTS. We provision, county pays the bill directly. County retains root SSH access; vendor handles hypervisor and provider backups.

Effort: half-day for a Linux-comfortable admin; two days for a learner. **Monthly:** \$6 (droplet) + \$1 (provider backups) + ~\$1 (AI) = \$8/mo. **Best for:** chambers that prefer "we know one person who can SSH in and fix things" over an in-house ops team.

3b. Scenario B — AWS / Azure / GCP IaaS

Profile: AWS EC2 t4g.small or Azure Standard_B1ms, Ubuntu image. Roughly \$15–25/month for the smallest instance the platform needs. County may already have an enterprise account, security baselines, and an existing landing-zone configuration to drop into.

Effort: half-day if the county has paved-road provisioning; one day otherwise. **Monthly:** \$15–25 (instance) + \$5 (EBS storage & snapshots) + ~\$1 (AI) = \$21–31/mo. **Best for:** counties standardized on a public-cloud vendor with existing tooling.

3c. Scenario C — County on-prem VMware / Hyper-V (Linux VM)

Profile: Ubuntu 24.04 LTS as a guest on county-managed hypervisor. 2 vCPU, 4 GB RAM, 30 GB disk is plenty. County handles host-level backups, patching cadence, and network access; we install the application stack inside the VM.

Effort: half-day to a day depending on how long VM provisioning takes through the county's request process. **Monthly:** incremental (counted as part of existing virtualization licensing). AI use still ~\$1/mo if outbound is permitted. **Best for:** counties with existing private-cloud capacity and policy preference for on-prem.

3d. Scenario D — Windows Server (IIS or Python service)

Profile: Windows Server 2022, IIS reverse-proxies to the Flask admin server, scraper runs as a Windows Service via NSSM or as a scheduled task via Task Scheduler. Chromium must be installed for Selenium; HTTPS via win-acme (alternative to certbot).

Effort: one to two days. The scraper depends on Selenium with headless Chromium — works on Windows but is less battle-tested than Linux. Chocolatey or winget for installs.

Monthly: incremental (existing Windows Server licensing) + AI ~\$1. **Best for:** counties whose IT shop is exclusively Windows and has no Linux administration capacity.

Sizing summary (all scenarios):

Resource	Minimum	Recommended	Why
CPU	1 vCPU	2 vCPU	Headless Chromium during scrape; 1-2 minute peak
RAM	2 GB	4 GB	Chromium memory footprint; Flask itself is tiny
Disk	10 GB	30 GB	Code under 100 MB; JSON files grow slowly; backups optional
Bandwidth	5 GB/mo egress	20 GB/mo	Static site; map/font CDN traffic served by 3rd parties
Inbound ports	443 only	443 + 22 (SSH limited)	80→443 redirect; SSH locked to admin IPs

4. Migration Plan

Ten phases, each independently verifiable. The walkthrough assumes Linux (scenarios A/B/C); Windows differences are called out where relevant. A Linux-comfortable admin completes phases 0–9 in a focused half-day.

Phase	What happens	Owner	Verifiable when
0. Decisions	Pick hosting scenario, domain name, DNS provider, who owns the bill. Generate Anthropic API key (optional).	County IT + chamber	Hosting account + domain registered
1. Pre-flight	Domain DNS access confirmed, SSH key pair created and saved, Anthropic key stored in password manager.	County IT	All credentials in a shared password vault
2. Provision	VM/droplet created, Ubuntu 24.04 LTS installed, security baseline applied (firewall, fail2ban, automatic updates), Python 3.12 / nginx / chromium installed.	County IT	<code>`ssh ncexp@server`</code> connects; <code>`nginx -v`</code> and <code>`python3 --version`</code> succeed
3. Deploy	Repository cloned to <code>/opt/ncexp</code> , Python venv created, requirements.txt installed, config.py populated with API key.	County IT or contractor	<code>`python server.py`</code> starts cleanly on port 5000
4. systemd service	ncexp.service unit file installed under <code>/etc/systemd/system</code> , enabled to auto-start on boot, auto-restart on crash.	County IT	<code>`systemctl status ncexp`</code> reports active (running)
5. nginx + TLS	nginx vhost reverse-proxies localhost:5000 to public 443. Certbot issues Let's Encrypt certificate; auto-renew cron installed.	County IT	<code>`curl https://ncexperience.county.gov`</code> returns the site; SSL Labs grade A
6. Schedule scrapes	cron entries (Linux) or Task Scheduler entries (Windows) added: nightly source refresh at 3 AM, AI categorize at 3:30 AM, daily backup at 4 AM.	County IT	<code>`/var/log/ncexp-scrape.log`</code> shows a successful 3 AM run
7. Backups	Provider snapshots enabled (DO/AWS) OR county VM backup policy applied (C). Daily JSON tarball mirrored to off-site (S3, county SAN, or scp).	County IT	Restore test: pull yesterday's tarball, verify events.json round-trips
8. Test checklist	Public site loads in all major browsers; admin Events Queue accepts approve/dismiss; AI Categorize button completes a small batch (skip if AI disabled); <code>/feed.rss</code> returns valid XML.	Chamber + IT	All seven items in the test sheet pass
9. Hand-off	Document handed to chamber: admin URL, log paths, common troubleshooting, escalation contact. County IT retains root.	County IT + chamber	Chamber operator runs through the operator_guide.pdf flow unassisted

Phase 6 — schedule example (Linux cron)

```
# Edit the ncexp user's crontab
sudo -u ncexp crontab -e
```

```
# Add three lines (UTC times – adjust for PT if server isn't on PT):
0 3 * * * cd /opt/ncexp && .venv/bin/python scraper/event_scraper.py >> /var/log/ncexp-scrape.log 2>&1
30 3 * * * cd /opt/ncexp && .venv/bin/python scraper/ai_categorize.py >> /var/log/ncexp-ai.log 2>&1
0 4 * * * tar czf /var/backups/ncexp-$(date +%Y%m%d).tgz /opt/ncexp/scraper_output
```

Phase 6 — schedule example (Windows Task Scheduler)

```
schtasks /Create /TN "NCExp-Scrape" /TR "C:\ncexp\.venv\Scripts\python.exe C:\ncexp\scraper\event_scraper.py" /SC DAILY /ST 03:00 /RU ncexp
schtasks /Create /TN "NCExp-AI" /TR "C:\ncexp\.venv\Scripts\python.exe C:\ncexp\scraper\ai_categorize.py" /SC DAILY /ST 03:30 /RU ncexp
```

5. Administration Responsibilities & Frequencies

Two roles maintain the platform: **chamber operator** (content curation, queue review) and **county IT** (server health, security patches, backups). Time estimates assume the platform is running normally; incidents are separate.

5a. Chamber operator tasks

Task	Frequency	Time	Tool
Review the pending Events Queue, approve / dismiss	2–3× per week	5–10 min	Admin → Events Queue tab
Run AI Categorize after a scrape	After each scrape	~30 sec (click + wait)	Admin → AI Categorize button
Click ■ Prune Past between scrapes if queue grows	Ad-hoc	30 sec	Admin → Events Queue → ■ Prune Past
Review the Suggestions tab (visitor-submitted ideas)	Weekly	10–15 min	Admin → Suggestions tab
Spot-check that scrapers are not returning 0 events	Weekly	2 min	Admin → Events Queue status bar
Publish a Suggested Experience (curated itinerary)	As inspiration strikes	15–30 min	My Itinerary → ■ Publish
Edit / delete a published Suggested Experience	Ad-hoc	5 min	Suggested Experiences tab → ✎ Edit
Add a new venue / experience	When a member opens	5 min	Admin → Experiences tab → + Add row
Update an experience's hours, photo, or notes	Ad-hoc	2 min	Admin → Experiences tab → inline edit
Add a new event-source URL	When discovered	10 min	Admin → Event Sources tab
Quarterly review of disabled scraper sources	Quarterly	30 min	Admin → Event Sources, re-test blocked
Annual full-catalog walk-through (verify URLs, hours)	Annual	4–6 hours	Admin → Experiences, row by row

Estimated steady-state chamber operator time: ~30 minutes per week during normal operation, plus a 4–6 hour annual review.

5b. County IT tasks

Task	Frequency	Time	Tool / location
OS security updates	Weekly (auto, monitored)	5 min review	`unattended-upgrades` log
Application updates (git pull from upstream)	Monthly or as needed	15 min	`git pull && systemctl restart ncexp`
Backup verification — pull last tarball and dry-run restore	Monthly	15 min	`tar tzf ncexp-YYYYMMDD.tgz`
TLS certificate expiry check (certbot auto-renews; verify)	Quarterly	5 min	`certbot certificates`

Task	Frequency	Time	Tool / location
Log rotation health	Quarterly	5 min	<code>`ls -la /var/log/ncexp-*.log`</code>
Disk usage review	Quarterly	5 min	<code>`df -h` / `du -sh /opt/ncexp/scrapper_output`</code>
User access review — SSH keys, admin URLs	Semi-annual	15 min	<code>`~/.ssh/authorized_keys`</code> ; admin host whitelist
DR / restore drill — full server rebuild from backup	Annual	2–4 hours	Snapshot → fresh VM → restore JSON
Anthropic API key rotation	Annual or on staff change	10 min	Anthropic console + <code>`config.py`</code>
Vendor / external-service review (still permitted? alternatives?)	Annual	30 min	This document, section 1

Estimated steady-state county IT time: ~30 minutes per month during normal operation, plus an annual DR drill.

6. Security & Compliance Posture

Control	How it's handled
Inbound network exposure	HTTPS only (port 443). SSH (22) restricted to admin IPs via firewall. No other listening ports.
TLS / certificates	Let's Encrypt via certbot; auto-renew cron. Strong cipher suite per Mozilla "intermediate" profile.
Secrets storage	API keys and credentials in `config.py` (file mode 0600, owned by service user). Not in git. Optional: load from environment variables.
Authentication	Visitor: none required. Admin URL: localhost-only by default OR `?admin=1` query parameter on a whitelisted hostname. For public admin, add HTTP Basic auth at the nginx layer (5-minute config).
Authorization	Single chamber operator role; no fine-grained RBAC. Add nginx Basic auth users for multiple operators if required.
Personal data (PII)	None collected. Itinerary save is opt-in localStorage on the visitor device — never leaves their browser. Suggestion form collects only the optional email the visitor types.
GDPR / CCPA scope	Effectively out of scope: no cookies, no PII storage. Suggestion-form emails (if entered) fall under standard records-retention policy.
ADA / WCAG	WCAG 2.1 AA on the demo. High-contrast text, keyboard-navigable, semantic HTML. Re-audit recommended after each major UI change.
Logging	`/var/log/ncexp-scraper.log`, `/var/log/ncexp-ai.log`, `journalctl -u ncexp` for the web server. No visitor-IP retention by default.
Patch cadence	OS: `unattended-upgrades` for security patches. Application: monthly `git pull` review. Python deps: quarterly `pip list --outdated`.
Incident response	Site is stateless apart from the JSON files; full restore from backup is < 30 minutes. Provider snapshot rollback covers OS-level compromises.
Supply chain	All Python deps pinned in requirements.txt. Frontend deps via CDN (Leaflet, fonts) — self-host option documented in section 7 if CDN use is restricted.

7. Flask Server — Security Review

The public site is plain HTML/JS and can be deployed to any web server with no special review. The **Flask admin server** is the only component that warrants security scrutiny — it executes Python, spawns subprocesses (headless Chromium for scrapes), holds API credentials, and writes to the local filesystem. This section enumerates the concerns and the mitigations available out of the box.

7a. What Flask does (in one paragraph)

A ~280-line Python script (`server.py`) using Flask. Listens on `localhost:5000` behind an nginx reverse proxy. Serves the static `index.html`; exposes a small JSON API used by the in-page admin tabs (Events Queue, AI Categorize trigger, Suggestions review, Scraper Sources). Stateless across restarts — every request reads/writes JSON files on disk; no database, no in-memory sessions.

7b. Concerns IT will raise (and pre-built mitigations)

Concern	Default state	Mitigation	Effort
Admin endpoints have no login wall	Gated by network only — listens on localhost; admin UI shown when host = 127.0.0.1 or URL contains <code>?admin=1</code> on a whitelisted hostname.	Add HTTP Basic auth at the nginx layer in front of <code>/admin*</code> paths. Optionally upgrade to SAML/OIDC via <code>oauth2-proxy</code> if the county has SSO.	5–30 min
Endpoints execute subprocesses (Python scraper, Chromium)	<code>/api/scrape</code> and <code>/api/ai/categorize</code> spawn child processes in a background thread on admin click.	Run Flask as an unprivileged user (<code>ncexp</code>) with a restricted shell; subprocesses inherit the same low-privilege identity. The only callable scripts are inside <code>/opt/ncexp/scraper/</code> , not arbitrary code.	15 min
Long-running Python process owns credentials and writes files	Anthropic API key in <code>config.py</code> (mode 0600). Writes to <code>scraper_output/*.json</code> .	systemd hardening: <code>NoNewPrivileges=true</code> , <code>ProtectSystem=strict</code> , <code>ProtectHome=true</code> , <code>PrivateTmp=true</code> , <code>ReadWritePaths=/opt/ncexp/scraper_output</code> , <code>CapabilityBoundingSet=</code> . Confines the process to its data directory.	30 min
Containerization / process isolation	Bare-metal Python process under <code>systemd</code> .	Optional: run in Docker/Podman with <code>--read-only root</code> , <code>--cap-drop=ALL</code> , volume-mount only <code>scraper_output</code> . Standard county-IT practice on many sites.	1–2 hours
Public suggestion form open to abuse	POST <code>/api/suggestions</code> accepts visitor input (name/email/description). Input length is capped server-side but no rate limit.	Add nginx <code>limit_req</code> zone — 30 req/min/IP on <code>/api/suggestions</code> . Optional: Cloudflare Turnstile (free, no cookies) on the form.	30 min
Python / Flask / Selenium / Chromium supply chain	Pip packages pinned in <code>requirements.txt</code> . Chromium installed via <code>apt</code> .	Schedule <code>pip-audit</code> weekly (or use Dependabot if mirrored to county GitHub). unattended-upgrades for OS + Chromium CVEs. Quarterly <code>pip</code> -outdated review.	15 min setup

Concern	Default state	Mitigation	Effort
No audit trail of admin actions	Admin clicks (approve/dismiss) are appended to events.json but not separately logged with operator identity.	Add nginx access log retention; if Basic auth is enabled, the username is logged per request. Optional: emit per-action audit entries to /var/log/ncexp-admin.log.	15 min
What if Flask itself is compromised?	Worst-case: attacker has read/write on scraper_output/ + can call the Anthropic API.	Provider snapshots provide hour-level rollback. Anthropic key has a \$5/month hard spending cap. No PII to exfiltrate. systemd hardening limits filesystem damage to the data directory.	baseline + monitoring

7c. The "split deployment" option for strictest policies

If county policy treats the dynamic Flask layer as too sensitive to host on its own server, the site can be split:

Public site	Static HTML/JS	Hosted on a CDN or county static-web bucket. Zero attack surface. No Python, no inbound port management, no patch cadence.
Admin layer	Flask + scraper	Hosted on a small internal VM the chamber operator reaches via VPN or county intranet only. Public internet never sees the Flask process.
Data hand-off	Generated artifact	Admin layer publishes its scraper_output/events.json to the public bucket on a cron (rsync, aws s3 cp, or equivalent). Visitors fetch it from the CDN.

Trade-off: the "Suggest a venue" public form needs SOME live endpoint — either keep a tiny suggestions-only Flask on a hardened sub-host, replace it with a static form-to-email service (Formspree, Tally, Microsoft Forms), or remove the public form entirely. Everything else works in the split model.

7d. What runs entirely without Flask

The visitor experience is HTML/JS only. With Flask off and only the static `index.html` + a periodically-updated `scraper_output/events.json` available, visitors get: theme browsing, vibe-pill filtering, day-by-day itinerary builder, opt-in localStorage save, share-link generation, map view, print, and Suggested Experiences browse. The only things that require Flask are the chamber-side admin tools and the public Suggest-a-venue submission form.

8. Policy-Block Impact Matrix

For each external service, what breaks if county policy disallows it and what the workaround is. Sites continues to function in nearly all cases; severity scales with how visitor-facing the lost feature is.

If this is blocked...	Impact	Workaround
Outbound HTTPS to api.anthropic.com	AI Categorize button stops working. Newly-scraped events keep their raw scraper-assigned tags — area/venue/quality refinement disabled. Operators must hand-tag events with bad metadata.	Hide the AI Categorize button in admin; rely on operator review. Lose ~20–30% of automated tagging accuracy. No site-visitor impact.
Outbound HTTPS to plausible.io	Aggregate metrics (visitor counts, itinerary build rate) unavailable. Site continues to work; chamber loses operational visibility into usage.	Self-host Plausible on the same VPS (Docker image, ~\$0 incremental). Or use the file-based access-log analytics built into nginx. Or disable analytics entirely.
Outbound HTTPS to fonts.googleapis.com (visitor browsers)	Custom fonts fall back to system serif/sans-serif. Site readable, slightly less branded look.	Self-host the Playfair Display + Josefin Sans WOFF2 files from /static/fonts and update the @font-face rules in index.html. ~20 KB extra payload per visitor.
Outbound HTTPS to cdnjs.cloudflare.com (Leaflet)	Map view button fails. Itinerary still works; visitors lose the "View on Map" feature.	Self-host Leaflet (one JS + one CSS file, ~150 KB total) from /static/leaflet/.
Outbound HTTPS to images.unsplash.com	~30 theme-card photos fail to load. Theme cards render with their gradient backgrounds; site still usable.	Download Unsplash photos once, commit to /static/photos/, update the THEMES[] array. ~3 MB one-time addition to the repo.
Outbound HTTPS to scraper source sites (KVMR, Trumba, etc.)	No new events arrive. Existing events keep showing until they expire; the catalog goes stale within ~2 weeks.	Operate in static-catalog mode: chamber adds events manually via the admin UI or the Suggestions form. Loss of ~95% of event volume.
Outbound HTTPS to acme-v02.api.letsencrypt.org (cert renewal)	TLS certificate eventually expires (90-day life). Site becomes inaccessible after expiry.	Use a county-internal CA, or a paid cert from a vendor the county already uses (DigiCert, Sectigo). Renewal becomes a manual quarterly task.
Inbound HTTPS (443) from public internet	Site is internal-only. Visitors outside the county network can't reach it.	Re-purpose as a kiosk / intranet tool. Defeats the public-facing visitor-planning value; not recommended unless that is the intent.
Cron / Task Scheduler permitted to run as a service account	Scrapes never run automatically. Queue stays empty. Auto-prune of past events never fires.	Operator manually clicks "Update Now" in the admin UI 1–3 times per week. Adds ~5 min/week to operator load.
Outbound SMTP (any provider)	No effect on this platform — we don't send email. Reserved for future alerting only.	N/A — feature not yet built.

The "everything off but the lights" scenario: with every optional outbound service blocked (no Anthropic, no Plausible, no CDNs, no scrape, no Let's Encrypt), the platform still functions as a manually-curated static catalog hosted on the county server with a county-internal certificate. Operator time roughly doubles (~1 hour/week instead of 30 min) because manual tagging and manual event entry replace the automated paths. No visitor-facing feature is broken; only the data-freshness rate drops.

9. Appendices

A. systemd unit file

```
[Unit]
Description=Nevada County Experience admin server
After=network.target

[Service]
Type=simple
User=ncexp
WorkingDirectory=/opt/ncexp
Environment="ANTHROPIC_API_KEY=sk-ant-..."
ExecStart=/opt/ncexp/.venv/bin/python server.py
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

B. nginx vhost (HTTPS, with redirect from HTTP)

```
server {
    listen 80;
    server_name ncexperience.county.gov;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name ncexperience.county.gov;

    ssl_certificate      /etc/letsencrypt/live/ncexperience.county.gov/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/ncexperience.county.gov/privkey.pem;
    include              /etc/letsencrypt/options-ssl-nginx.conf;

    # Strong default headers
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header X-Content-Type-Options    "nosniff" always;
    add_header X-Frame-Options           "SAMEORIGIN" always;

    location / {
        proxy_pass      http://127.0.0.1:5000;
        proxy_set_header Host          $host;
        proxy_set_header X-Real-IP     $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

C. Daily JSON backup script

```
#!/bin/bash
# /opt/ncexp/bin/backup-daily.sh – run by root cron at 04:00
set -euo pipefail
STAMP=$(date +%Y%m%d)
DEST=/var/backups/ncexp
mkdir -p "$DEST"
tar czf "$DEST/ncexp-$STAMP.tgz" /opt/ncexp/scrapper_output /opt/ncexp/config.py
# Retain 14 days
find "$DEST" -name 'ncexp-*.tgz' -mtime +14 -delete
# Optional: mirror to county SAN / S3
# rsync -a "$DEST/" backupserver:/backups/ncexp/
```

D. Reference URLs

Document / artifact	Where
Live demo (currently on GitHub Pages)	https://liammlrb-eng.github.io/nevada-county-experiences/
Source repository	https://github.com/liammlrb-eng/nevada-county-experiences
Operator day-to-day guide	operator_guide.pdf (in this bundle)
Long-form pitch & UX rationale	demo_pitch.pdf (in this bundle)
Slide deck	demo_pitch.pptx (in this bundle)

Document version: generated automatically from `generate_it_brief.py`. Regenerate after any change to dependency list, hosting scenarios, or migration plan.